neander-assignment

Initial plan for the take-home assignment: An App Platform for Coding Agents
13 October 2026

What I'm building

I'm building a small local platform that a coding agent can use to create, run, update and inspect simple data-backed apps. The agent writes a JSON contract and a Python file of handlers, deploys it, runs it on localhost, calls it, and inspects what happened. Every app runs on the same runtime and keeps its data in its own SQLite file.

I'll write it in Python 3.11 using only the standard library at runtime (`sqlite3`, `http.server`, `argparse`), with pytest for tests, so setup is a single `pip install -e .`

How an app is defined

An app is a folder with two files. `app.json` declares the data fields and names each handler as read or write. `handlers.py` has one Python function for each handler. For example:

```
{
  "name": "notes",
  "collections": {
    "notes": {"owner": {"type": "string"}, "title": {"type": "string"}}
  },
  "handlers": {"create_note": "write", "list_notes": "read"}
}
```

```
def list_notes(ctx, args):
    return ctx.db.find("notes", where={"owner": ctx.caller})
```

Field types are kept to `string`, `integer`, `number` and `boolean`, and every write is validated against the declared fields. Each handler gets `ctx.caller` (the declared identity), `ctx.db` (the data handle) and `ctx.log()`. The data handle only has `get`, `find`, `count`, `insert`, `update` and `delete`.

I'm keeping the contract in JSON rather than in Python code so the platform can validate it and show it in `inspect` without running the app's code, even when that code is broken.

How it runs

- Write handlers run in a single transaction, so a handler that fails halfway leaves no partial writes.
- Read handlers get a read-only data handle. Any write through it fails with a structured `READ_ONLY_VIOLATION` error.
- The server is single-threaded, so writes are naturally sequential and no locking code is needed.
- Each app has its own SQLite file, so data is kept separate by how it's stored, not by a filter someone could forget.
- Every call, error and deploy attempt is logged so `inspect` can show recent activity.

Updates and rejected definitions

Updates go through an explicit `neai deploy`. Before anything is activated, the platform checks the manifest, imports the handler code, confirms every declared handler exists, and checks that the schema still fits the data already stored. If any check fails, the deploy is rejected with a clear error code and hint, the data is untouched, and the last valid version keeps running. `neai inspect` shows why the latest definition was rejected. On restart, the server falls back to the last valid version.

For schema changes, I'll accept a change only if all existing data is still valid under the new schema (for example, adding an optional field), so no migration logic is needed.

CLI

Every command prints JSON, so an agent can read the result directly.

- `neai new <name>` – create a skeleton app
- `neai deploy <dir>` – validate and activate a new version
- `neai run <dir>` – serve the app on localhost
- `neai call <app> <handler> --as <caller>` – call a handler
- `neai inspect <app>` – show the version, handlers, data, recent logs and last rejection

Example apps

- **Notes:** callers can create, list and update their own notes. Every lookup is filtered by the caller, and another caller's note looks exactly like one that doesn't exist.
- **Feedback:** callers create posts and vote on them. Counts come from a votes collection, and a repeat vote by the same caller on the same post isn't counted twice.

Tests, demo and docs

Tests will cover validation, caller scoping, read/write restrictions, data separation, persistence across a restart and a code update, and rejected definitions. A demo script will run both apps, restart the server to show the data persists, deploy a code update, and deploy a broken definition to show it being rejected. `AGENTS.md` will explain how a coding agent should build and run an app, and before submitting I'll check it by having a fresh agent build a third app using only those docs.

Main trade-offs

- A JSON contract rather than decorators, so it can be checked without running app code.
- JSON records in SQLite rather than a table per collection, to avoid migrations.
- Explicit deploys instead of file watching, so the agent always gets a clear answer.
- The standard library instead of FastAPI or Click, to keep setup to one command.

The main known limitation is that handlers have no timeout, so a handler stuck in a loop would block the server until it's restarted. Migrations, authentication, concurrent writes, multiple apps per server and a UI are out of scope.